

Assignment 1 Report

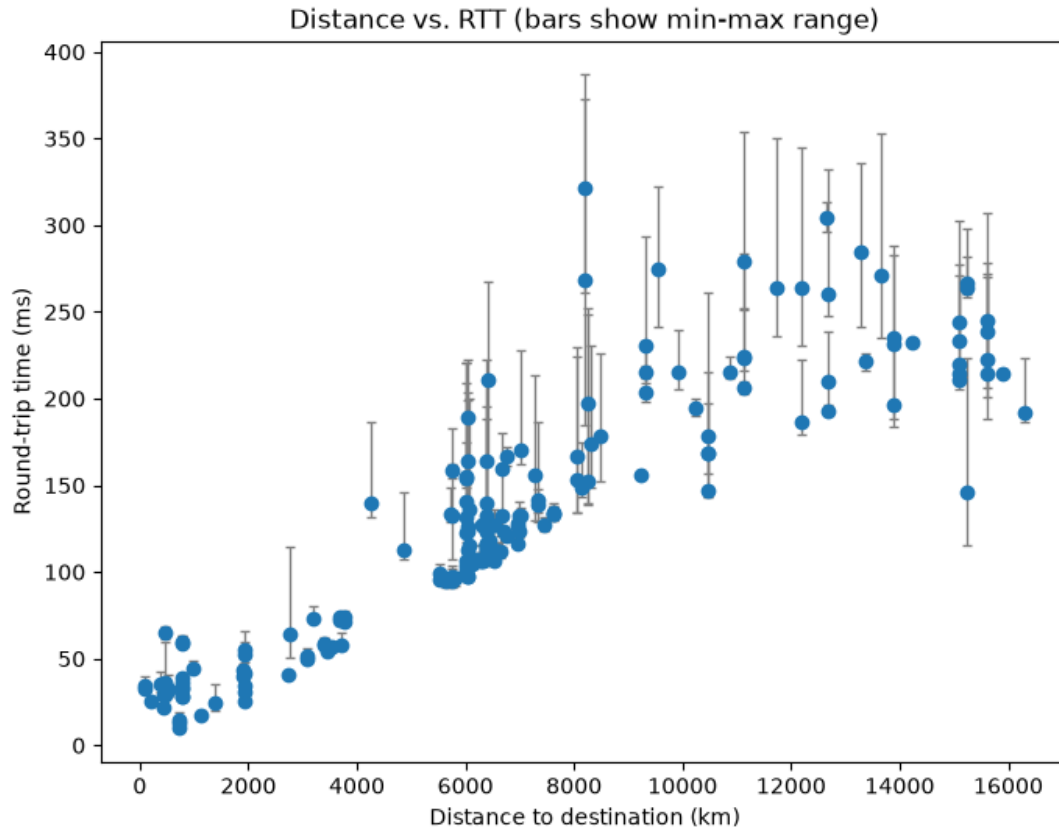
AI Acknowledgement: We used Claude in this assignment to assist with code reviews, merge conflicts, as well as generating code on the function level.

Repo: <https://github.com/CS422-Networks-Group/HW1> (branch [submission](#))

1 (c) Does distance relate to RTT?

Yes, RTT (especially min RTT) correlates strongly with distance. Close servers (Chicago, Washington, Toronto, <1000 km) had min RTTs of 9-32 ms; far ones (Kediri, Floreal, ~15,000-16,000 km) had 213-261 ms.

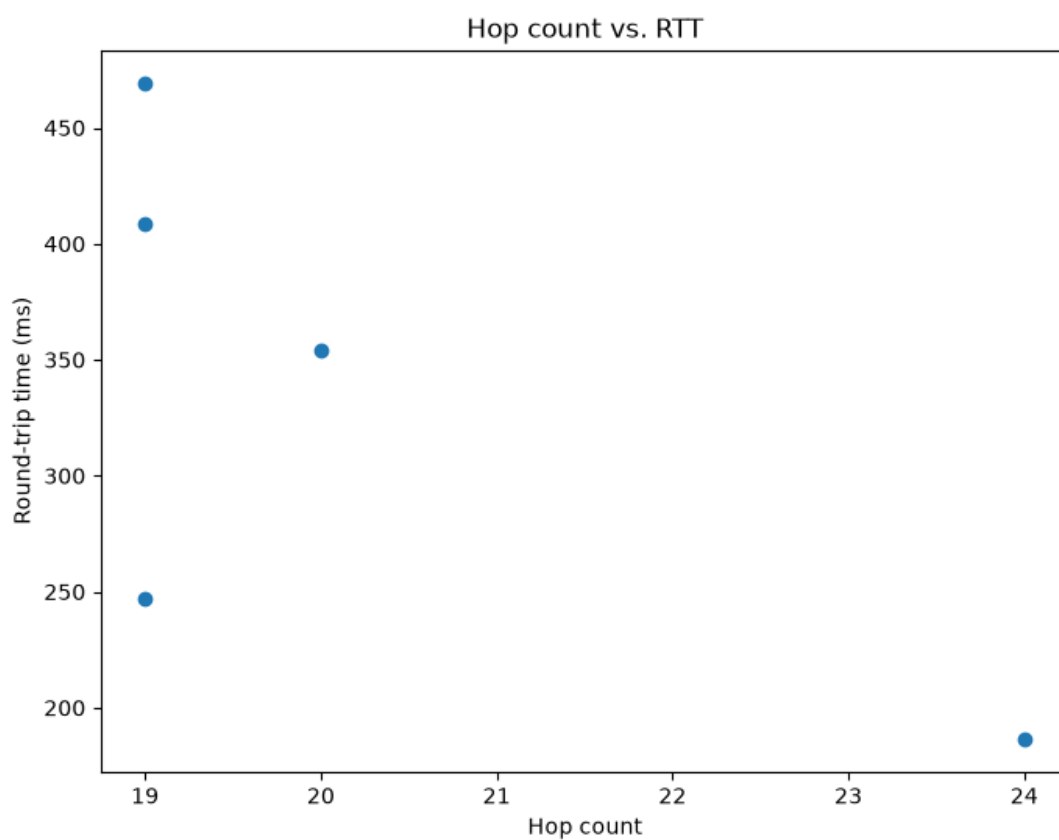
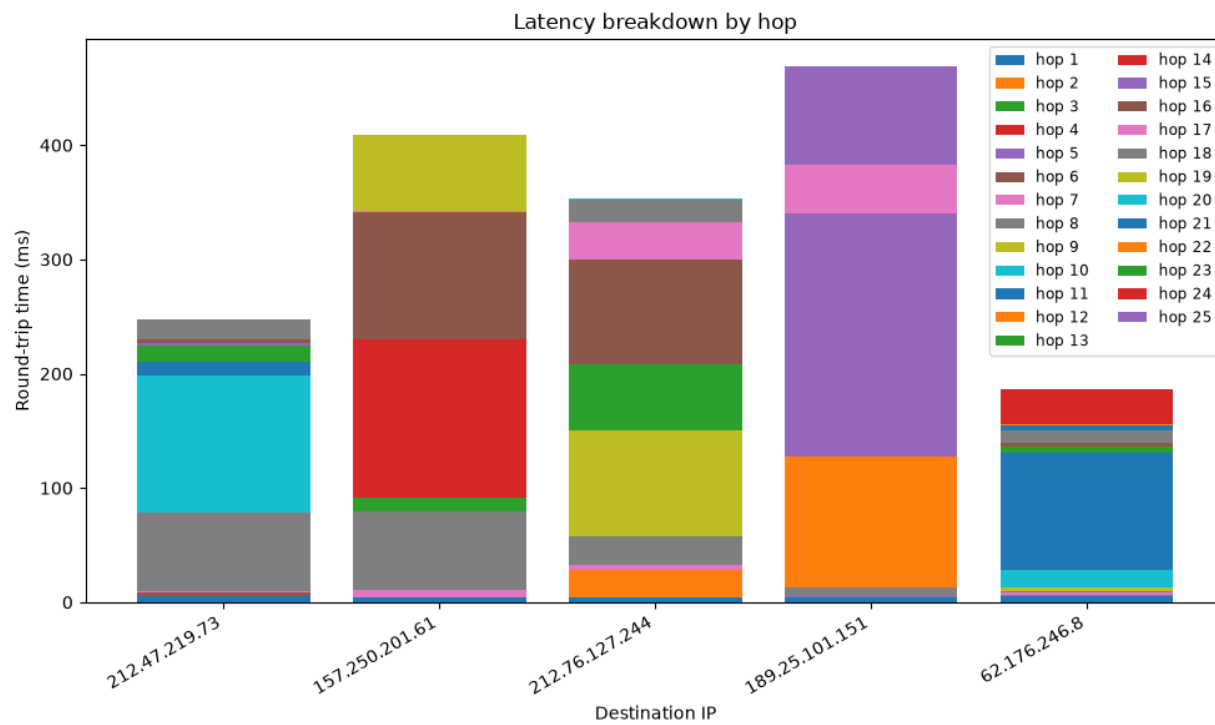
Max RTT correlates much less with distance, with several notable high outliers despite their associated min RTTs seeming "normal" relative to the distance. This likely means that congestion/queuing along the path is more relevant than distance for jitter. At almost the exact same distance (13,887 km), Auckland spiked to 99-101 ms of jitter between max and min RTT on both servers measured there, while Christchurch, at an identical distance, showed almost none (4.9 ms).



Generated by `plot_distance_vs_rtt` in `src/visualize.py`, called from `main()` in `src/main.py`.

2 (d) Explain your observations from these plots. How does hop count relate to RTT?

Hop count barely predicts RTT at all. Three of our five destinations tied at 19 responsive hops (bwtest.linxtelecom.com, dfw.speedtest.is.cc, 189.25.101.151) still spread from 247 ms to 469 ms in total RTT, and the path with the *most* hops (a210.speedtest.wobcom.de, 24 hops) had the *lowest* RTT (186 ms) of all five. The stacked bar chart shows why: most hops -- the local network and the first couple of transit hops -- each add only a few ms, and total RTT is instead set by one or two slow hops later in the path. For example, on the path to 189.25.101.151, a single hop (62.216.131.29) adds 213 ms by itself -- nearly half of that path's entire 469 ms RTT. So RTT depends much more on which provider/backbone a path happens to cross than on how many hops it takes to get there.



Generated by [plot_latency_breakdown](#) and [plot_hopcount_vs_rtt](#) in `src/visualize.py`, called from [main\(\)](#) in `src/Latency_breakdown.py`.

References

Repo: <https://github.com/CS422-Networks-Group/HW1> - all code below is on branch [submission](#).

Part 1 - ping + distance vs. RTT ([src/main.py](#)):

- [process_csv](#) - resolves each target's location; drops rows whose host can't be resolved, and skips a host that resolves but has no entry in the IP2Location DB, instead of failing the whole run.
- [execute_ping_tests](#) - corner case: if a target doesn't respond to ping (no RTT summary line), records None for that row rather than crashing.
- [main](#) - single entry point: takes the input CSV of targets as a CLI arg, thread-pools the ping tests, writes the results CSV, and generates the distance-vs-RTT PDF, all in one run (`python3 src/main.py <targets.csv>`).

Part 2 - traceroute + latency breakdown ([src/latency_breakdown.py](#)):

- [_parse_traceroute_output](#) - corner case: filters out non-responsive hops (e.g. * *) instead of treating them as zero-latency.
- [get_latency_breakdown](#) - corner case: an unresolvable host or a failed traceroute call returns an empty result for that target instead of aborting the run.
- [to_dataframe](#) - converts per-hop RTTs into the hop-to-hop deltas used for the plots.
- [main](#) - single entry point: takes the same CSV of targets, randomly samples 5, traceroutes each, and writes the JSON results plus both PDFs in one run (`python3 src/latency_breakdown.py --input <targets.csv>`).

Plotting ([src/visualize.py](#)):

- [plot_distance_vs_rtt](#)
- [plot_latency_breakdown](#)
- [plot_hopcount_vs_rtt](#)